前面 4 页只填空，
黑色字不能改动！

湖南工業大學
HUNAN UNIVERSITY OF TECHNOLOGY

（2025 届）
本科毕业论文（设计）资料

题目名称行根据需要增删，行距固定值 20 磅，居中，四号字，加粗

课题名称：

学　　院：计算机学院

专　　业：

学生姓名：

班　　级：　　学　号：

指导教师　　职　称：

职　称：

班级举例：
计算机 210×；
软件工程 210×；
网络工程 210×；
物联网 210×；
智能科学 210×；
人工智能 210×
不带“班”字。
凡属班级名称都按此填写。

根据实际指导教师数增删行，联合毕业设计的需在校内导师之后加入校外导师的信息。

湖南工业大学 教务处

2025 届

本科毕业论文（设计）资料

第一部分　毕业论文

重要提示：

- 本模板中的**蓝色**文字为格式说明，在参阅后自行删除。
- 本模板中的**红色**文字为内容说明，在参阅后自行删除或修改为论文详细内容。详细内容要求请参阅毕业设计教学大纲。
- 建议用 Word 编辑，WPS 可能会导致格式异常。
- 建议以填空方式直接在模板里写作，以避免繁杂的格式调整。注意发挥格式刷的作用，从外部粘贴文字时，要“匹配目标格式”，不要“保留源格式”。
- 本模板中，凡没指明字体的地方，中文都默认为宋体，非中文都默认为 Times New Roman。一次性设置非中文字体的方法是：ctrl+A 选中全文，选 Times New Roman（中文会保持原字体）。
- 一切通过增加空格或空行来对齐文本的操作，都是不可取的。

本页所有格式与第 1 页相同，不同的只是文字内容，因此不重复说明

（2025 届）

本科毕业论文（设计）

课题名称：

学　　院：计算机学院

专　　业：

学生姓名：

班　　级：　　学号：

指导教师：　　职称：

　　职称：

2025 年 5 月

湖南工业大学

本科毕业论文（设计）诚信声明

本人郑重声明：所呈交的毕业论文（设计），题目《……此处填写论文题目……》是本人在指导教师的指导下，进行研究工作所取得的成果。对本文的研究作出重要贡献的个人和集体，均已在文章以明确方式注明。除此之外，本论文（设计）不包含任何其他个人或集体已经发表或撰写过的作品成果。本人完全意识到本声明应承担的责任。

作者签名：

日　　期：　　年　　月　　日

摘 要

论文围绕一个由大语言模型智能体驱动的 Unity 二维像素风小镇经营系统展开研究。系统面向传统经营模拟游戏中非玩家角色行为模板化、经济反馈链条单一、玩家与智能角色互动缺少长期状态记忆等问题，提出“决策层与执行层分离”的设计方案：后端采用 Python 异步运行时维护世界状态、角色记忆、动作校验、市场结算和智能体决策，前端采用 Unity 构建像素风场景、角色动画、商店界面和玩家操作入口，两层通过 WebSocket 协议形成回合闭环。研究重点包括基于 LLM（Large Language Model，大语言模型）的 Agent（Autonomous Agent，自主智能体）计划、行动、执行、反思流程，动作注册表与规则校验机制，玩家商店定价与进货机制，以及均值回归价格模型和决策点反制机制。实现结果表明，该系统能够支持多名 AI（Artificial Intelligence，人工智能）角色在共享市场中围绕生存、交易和收益进行持续决策，同时允许玩家以商店店主身份调节商品价格、库存和现金流，使经营模拟从单向管理扩展为人与智能体之间的动态博弈。系统在结构上具有较好的可扩展性，能够继续增加商品、事件、角色技能和新的决策动作；在实践上也证明了大语言模型智能体与轻量级游戏运行时结合的可行性。

关键词：大语言模型智能体；Unity 二维游戏；多智能体系统；经营模拟；WebSocket

ABSTRACT

This thesis studies the design and implementation of a Unity two-dimensional pixel-art town management system driven by large language model agents. To address scripted non-player character behavior, weak economic feedback loops, and limited long-term interaction in traditional simulation games, the system adopts a separated architecture with a Python decision layer and a Unity execution layer. The decision layer maintains world state, agent memory, action validation, market settlement, and turn scheduling, while the execution layer presents the town scene, character movement, shop interface, animation feedback, and player operations. The two layers communicate through WebSocket messages and form a closed turn-based loop.

The core work includes an LLM-based plan-act-execute-reflect agent workflow, an action registry with deterministic validators, a shop management phase controlled by the human player, a mean-reverting stochastic price model, and an agent decision-point mechanism that can influence market information and price locking. The implemented system supports multiple AI agents that make continuous decisions around survival, trading, inventory, and wealth accumulation. Meanwhile, the player acts as the shop owner and manages pricing, restocking, and cash flow, turning the game into an asymmetric interaction between human operation and autonomous agents. The result indicates that combining large language model agents with a lightweight game runtime is feasible for small-scale interactive simulation, and the architecture can be extended with additional items, events, agent skills, and action types.

Keywords: large language model agent; Unity two-dimensional game; multi-agent system; business simulation; WebSocket

目　录

第 1 章 绪论

1.1 课题背景与意义

1.1.1 研究背景

1.1.2 研究意义

1.2 国内外研究现状

1.3 关键技术分析与比较

1.4 总体方案

1.5 系统特色与创新之处

第 2 章 系统分析

2.1 设计系统时考虑的相关因素

2.2 项目成本分析

2.3 需求分析

2.3.1 功能需求分析

2.3.2 非功能需求分析

2.4 可行性分析

2.4.1 技术可行性分析

2.4.2 操作与维护可行性分析

第 3 章 系统概要设计

3.1 系统总体架构设计

3.1.1 决策层与执行层分离设计

3.1.2 通信协议与回合闭环设计

3.2 功能模块设计

第 4 章 系统详细设计与实现

4.1 决策层详细设计与实现

4.1.1 多智能体运行时设计

4.1.2 动作空间与校验机制设计

4.2 市场与玩家商店系统实现

4.3 Unity 执行层详细设计与实现

4.4 系统稳定性与异常处理

第 5 章 系统部署与维护

5.1 系统安装与部署

5.1.1 决策层部署

5.1.2 执行层部署

5.2 系统测试

5.3 系统维护

5.4 项目管理与迭代

结论

参考文献

第 1 章 绪论

本章对应课题调研与方案提出部分，重点说明课题背景、研究意义、国内外研究现状、关键技术路线和总体设计方案。由于本系统同时涉及游戏开发、智能体运行时、多智能体交互和小型经济系统，因此调研对象不仅包括 Unity 游戏开发，也包括 LLM 智能体、多智能体强化学习、游戏智能体和虚拟仿真交互等相关研究。

1.1 课题背景与意义

1.1.1 研究背景

随着生成式人工智能的发展，LLM 已经从单轮问答工具逐渐扩展为能够进行任务规划、工具调用、环境交互和记忆维护的智能体核心。生成式智能体研究表明，基于自然语言记忆、反思和计划的智能体能够在沙盒环境中产生较可信的日常行为和社会互动[1]。国内关于大语言模型智能体的综述也指出，任务规划、决策执行和评估机制已经成为智能体落地的关键环节[2]。ReAct、Reflexion、Voyager 等研究进一步说明，LLM 在与外部环境连接后，不仅能够生成文本，还能够根据环境状态提出行动、接收执行反馈并修正后续策略[3-5]。这些研究为游戏中的 NPC（Non-Player Character，非玩家角色）行为设计提供了新的技术路径。

传统经营模拟游戏通常依赖预设脚本、行为树或有限状态机控制 NPC 行为。该方式稳定、可控、易调试，但也存在角色反应固定、长期目标单一、交互内容重复等问题。对于小镇经营、市场交易、生存消耗等具有动态反馈的玩法，固定脚本难以体现玩家操作与 NPC 策略之间的复杂互相影响。若将 LLM 智能体引入游戏运行时，则可以让角色依据当前资金、库存、饱食度、水分值、精神值、价格情报和历史行动进行决策，从而提升行为多样性和系统可演化性。

本课题的工作台项目是一个 Unity 二维像素风小镇经营系统。项目早期以多智能体生存与交易模拟为核心，四名 AI 角色在小镇中移动、买卖商品、使用补给并尝试积累资金。后续迭代中，系统重心转向“人类店主与 AI Agent 之间的经营博弈”：玩家经营小镇中唯一商店，通过定价、进货和现金流管理影响 AI 的生存与收益；AI 角色则通过计划、行动、交易和决策点机制对玩家形成反制。该设计把玩家从旁观者变成市场规则的一部分，也使系统从单纯智能体观察平台转变为可交互的经营游戏。

1.1.2 研究意义

第一，课题具有面向产品化经营模拟游戏的应用价值。传统经营模拟游戏通常依靠固定脚本维持 NPC 行为，玩家在熟悉规则后容易形成稳定套路，长期留存依赖持续增加内容。本系统把 AI Agent 引入商店经营场景，使角色能够基于资金、库存、生存属性、市场价格和历史行为持续调整策略。对于可落地产品而言，这种方式能够提升单局差异、降低纯脚本内容消耗，并为后续运营中的角色扩展、事件扩展和经济参数调优提供更高弹性的系统基础。

第二，课题具有可控 AI NPC 商品化的工程意义。面向真实游戏产品，AI NPC 不能只追求“会说话”，还必须满足状态安全、行为可解释、成本可控和可测试等要求。本系统把模型输出限制为结构化动作提案，再通过确定性代码进行合法性校验和世界状态更新。

LLM 负责计划和提出行动，规则系统负责裁决行动边界，Unity 负责执行与展示。这种职责分离能够降低模型不确定性对游戏经济和角色状态的破坏，也便于后续接入日志分析、自动化回放和运营调参。

第三，课题具有服务端驱动型游戏架构的落地意义。系统采用 DecisionLayer 与 ExecutionLayer 双层架构，将世界规则、市场结算、智能体决策和可视化表现拆分到不同层次中。该结构接近商业游戏中“服务端维护权威状态、客户端负责表现与输入”的基本思路：后端可以统一控制经济结算和 AI 行为，前端可以专注场景、动画和交互体验。对于后续扩展为多人观看、云端推理或内容运营工具，该分层能够减少重构成本。

第四，课题具有小规模 AI 原生游戏原型验证意义。系统不是把大模型作为外部聊天窗口，而是把智能体嵌入经营规则、市场价格、角色生存和回合结算之中。该原型能够验证 AI Agent 是否可以成为玩法循环的一部分，并为后续开发更完整的 AI 经营模拟产品提供技术路径、风险边界和迭代依据。

1.2 国内外研究现状

国外关于 LLM 智能体的研究发展较快。Park 等提出的 Generative Agents 使用记忆流、反思和计划机制模拟可信人类行为，证明大语言模型可以成为虚拟角色行为生成的核心[1]。Voyager 在 Minecraft 环境中引入自动课程、技能库和迭代提示机制，使智能体通过持续探索积累可复用技能[5]。ReAct 将推理轨迹与动作生成交替组织，使模型能够在推理过程中调用外部环境信息，从而提升可解释性和任务完成率[3]。Reflexion 使用语言反思替代权重更新，把失败反馈写入记忆，以改善后续决策[4]。这些工作共同说明，LLM 智能体的关键不只是模型能力，还包括记忆组织、环境反馈、行动约束和失败恢复。

在提示推理方面，Chain-of-Thought 证明中间推理步骤能够显著提升大模型在复杂推理任务中的表现[7]，Tree of Thoughts 进一步将推理扩展为多分支搜索和候选路径评估[8]。Toolformer 表明模型可以学习何时调用外部 API（Application Programming Interface，应用程序编程接口）并结合工具结果完成任务[9]。SayCan、Inner Monologue 和 LLM-Planner 等 embodied agent 研究强调，语言模型的计划必须受到环境状态、技能集合和可执行性的约束[11-13]。这些研究对本系统具有直接启发：智能体不应直接“改写世界”，而应提出能够被运行时验证和执行的动作。

在多智能体系统方面，AutoGen 通过多智能体会话框架支持复杂任务协作[14]，LLM-MA（Large Language Model based Multi-Agents，基于大语言模型的多智能体）综述总结了大语言模型多智能体系统在角色画像、通信、协作和仿真方面的进展与挑战[15]。Frontiers of Computer Science 上的 LLM 自主智能体综述提出了感知、记忆、规划、行动和评估等统一构成要素[17]，记忆机制综述进一步说明了短期记忆、长期记忆和外部记忆对智能体连续决策的重要性[18]。多智能体深度强化学习综述则指出多智能体环境普遍面临非平稳性、信用分配、可扩展性和部分可观测等问题[19]。本系统虽然没有采用深度强化学习训练策略，但其游戏世界同样存在多个主体在共享市场中同步改变环境的问题，因此需要通过动作校验、交易震荡保护、回合结算和玩家阶段同步来控制复杂性。

国内研究同样关注 LLM 智能体、多智能体强化学习和 Unity 应用。郭先会等对基于大语言模型的智能体构建进行了综述，指出任务规划、决策执行和测评方法是当前研究重点[2]。李明阳等、王若男等、项凤涛等、陈卓然等分别从算法分类、学习机制、群体视角和理论应用层面对多智能体强化学习进行了综述[6,10,16,30]。闫超等关注多智能体深度强化学习的可扩展性和可迁移性问题[20]，田琪等讨论了通信协作机制在多智能体强化学习中的作用[21]，薛宏涛等较早系统梳理了多智能体系统体系结构与协调机制[22]。这些研究说明，多智能体系统的核心难点不只是单体智能，还包括多个主体之间的协调、竞争、通信和稳定性。

在 Unity 游戏与仿真实现方面，国内已有大量实践研究。叶现勇等基于 Unity3D 实

现汉字游戏，强调游戏化学习、场景交互和界面组织[23]。刘宁晖、宋瑾钰在文字冒险游戏中融合 Unity、三维场景漫游和语音交互，体现了游戏引擎在叙事互动中的灵活性[24]。秦思、侯建峰研究了基于 Unity3D 的体感游戏实现[25]，余欣婷等讨论了移动端三维休闲游戏的策划、美术和交互功能[26]，刘梦颖、马宏琳围绕 Unity 冒险游戏设计了客户端、网络和场景模块[27]。此外，新能源汽车虚拟仿真和仿真教学系统相关研究说明 Unity 也适合用于交互式训练、流程展示和模拟系统开发[28-29]。与这些研究相比，本课题的特点在于把 Unity 前端与 LLM 智能体后端进行松耦合连接，使游戏表现层和智能决策层分别演进。

综合来看，已有研究为本课题提供了三方面基础：一是 LLM 智能体的记忆、计划、行动和反思框架；二是多智能体系统在共享环境中的协调与稳定性问题；三是 Unity 在游戏交互、界面搭建和仿真展示中的成熟工程能力。本课题的差异化工作在于将三者结合到一个可运行的二位像素风小镇经营系统中，并围绕玩家商店阶段设计人类玩家与 AI 智能体的市场博弈闭环。

1.3 关键技术分析与比较

本系统涉及的关键技术包括 Unity 二维游戏开发、Python 异步运行时、WebSocket 通信、LLM 智能体决策、动作注册与校验、市场价格模型和玩家商店 UI。各项技术的选型需要同时考虑可控性、开发成本、运行稳定性和毕业设计展示效果。

Unity 适合承担 ExecutionLayer。该引擎提供二维精灵、Tilemap、Animator、Canvas、TextMeshPro、输入系统和资源管理能力，能够快速搭建像素风小镇、角色移动、商店交互、弹窗反馈和回合动画。已有 Unity 游戏与仿真实践表明，Unity 在交互场景组织、用户界面搭建和虚拟环境展示方面具有较成熟的工程基础[23-24,28]。与直接使用网页 Canvas 相比，Unity 在角色动画、场景层级、碰撞和像素风素材管理方面更接近游戏项目真实开发流程；与三维引擎复杂场景相比，二维像素风能够把设计重点集中在系统架构和智能体交互上。

Python 适合承担 DecisionLayer。智能体决策、OpenAI 模型调用、JSON（JavaScript Object Notation）结构化解析、异步 WebSocket、市场结算和数据文件加载都可以用 Python 高效实现。项目没有引入完整后端 Web 框架，而是直接使用 asyncio、websockets、数据类和自定义运行时。这种方式减少了框架学习成本和中间抽象，也更便于控制智能体主循环、动作执行回执和失败恢复。

WebSocket 适合连接决策层和执行层。HTTP（HyperText Transfer Protocol，超文本传输协议）请求适合一次性数据查询，但本系统需要后端主动向 Unity 推送动作命令、市场信息和回合开始消息，也需要 Unity 回传动作完成状态和玩家商店提交结果。WebSocket 可以保持长连接，支持双向消息和异步等待。系统在协议层为每个动作生成 action_id，并等待 Unity 回传 complete 消息，形成轻量级 RPC（Remote Procedure Call，远程过程调用）语义。

LLM 智能体适合用于高层决策，而不适合直接修改游戏状态。模型擅长根据角色身份、资源状态、历史记忆和市场信息提出行动计划，但模型输出存在不确定性。自主智能体研究通常把规划、记忆、行动和反馈视作核心组成部分[17-18]，ReAct 等方法也强调推理与行动应在外部环境反馈中交替进行[3]。因此系统把 LLM 输出视作“动作提案”，再由动作注册表和校验器判断是否允许执行。该设计对应“生成式智能与确定性规则结合”的思路：模型负责灵活性，程序负责边界。

市场模型采用均值回归和对数噪声。若价格完全固定，交易策略会过于单调；若价格完全随机，玩家和 AI 很难形成可解释策略。系统采用对数价格上的离散均值回归过程，使价格围绕基础价格波动，同时保持正值和品类差异。消耗品波动较小，适合维持生存；贵重品波动较大，适合投机交易。这一模型参数少、可解释性强，能够支撑小型经营模拟。

表 1.1 对关键技术进行比较。

技术方向	可选方案	本系统采用方案	采用原因
游戏表现层	Web 前端、Godot、Unity	Unity 二维	像素风场景、动画和 UI 支持成熟
决策运行时	游戏内脚本、后端框架、自定义 Python	自定义 Python 异步运行时	便于模型调用、状态管理和协程调度
通信方式	HTTP、Socket、WebSocket	WebSocket	支持长连接、双向消息和动作回执
智能体控制	行为树、强化学习、LLM 智能体	LLM 智能体加规则校验	兼顾行为多样性与状态安全
市场模型	固定价格、纯随机、均值回归	对数均值回归	兼顾波动性、稳定性和可解释性

1.4 总体方案

本系统总体方案是构建一个“后端决策、前端执行、玩家商店阶段插入回合循环”的双层架构。DecisionLayer 负责世界状态、Agent 运行时、市场结算、动作校验和模型调用；ExecutionLayer 负责 Unity 场景、角色实体、移动动画、商店 UI、状态弹窗和玩家输入。两层通过 WebSocket 交换 JSON 消息。

系统流程如图 1.1 所示。

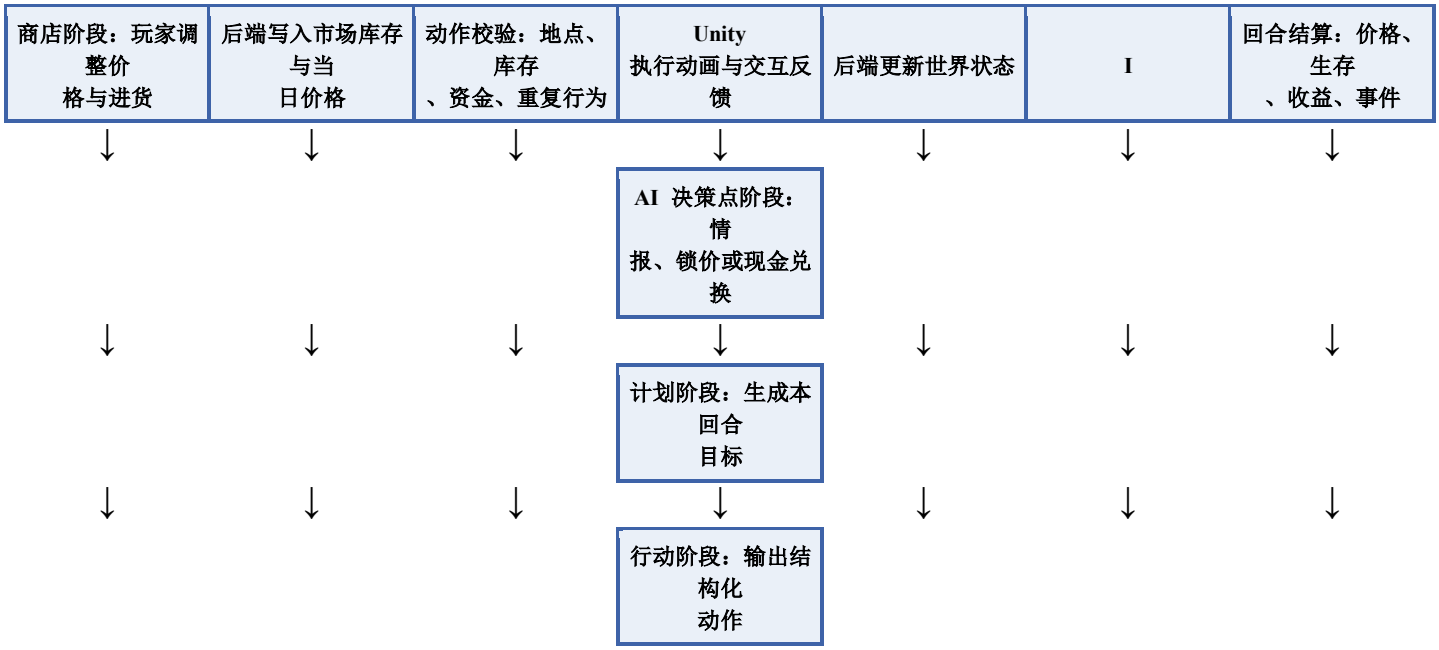

图 1.1 系统总体流程图

在该流程中，玩家每回合首先获得市场信息并提交经营方案。后端根据玩家提交结果更新市场库存、价格和玩家现金。随后 AI 智能体先进行决策点选择，再根据观察快照生成计划和动作。动作进入注册表后，由校验器判断是否符合地点、库存、资金和行为保护规则。通过校验的动作被发送到 Unity，Unity 执行动画后回传完成消息。所有 AI 结束行动后进入回合结算，系统推进日期并重新开启商店阶段。

该方案把玩家与 AI 放在同一个市场环境中。玩家经营商店不只是 UI 操作，而是直接改变 AI 购买、出售、补给和投机的外部条件；AI 的锁价和情报机制也不只是文本结果，而会真实影响下一日价格和玩家定价空间。因此系统形成“玩家策略影响 AI，AI 决策反制玩家，市场结算反馈双方”的闭环。

1.5 系统特色与创新之处

本系统的特色主要体现在以下五个方面。

第一，系统采用人类店主与 AI 智能体的不对称博弈设计。玩家掌握商店库存、价格和现金流，AI 掌握行动计划、交易策略和决策点资源。玩家胜利条件不是简单利润最大化，而是在自身资金达到目标之前，避免任何 AI 死亡或率先达到目标资金。这种目标设计迫使玩家在利润、生存供给和 AI 财富增长之间保持平衡。

第二，系统采用 DecisionLayer 与 ExecutionLayer 分离架构。后端只维护规则和状态，不直接依赖 Unity 对象；前端只执行表现和输入，不承担大模型决策。该分层降低了耦合度，使得后端可以独立调试动作逻辑和市场模型，前端可以独立迭代 UI、动画和场景表现。

第三，系统将 LLM 输出纳入确定性动作管线。智能体的 act 阶段必须输出结构化动作，ActionExecutor 再根据动作注册表查找 handler 和 validators。模型可以提出买、卖、移动、使用、休息、等待等动作，但是否执行由程序判断。该方式保留 LLM 的灵活性，同时避免非法动作破坏世界状态。

第四，系统设计了 AI 决策点机制。AI 可消耗决策点获取下一日价格情报、锁定商品价格或兑换现金。锁价会限制玩家对指定商品的价格调整，情报会改变 AI 的交易判断。该机制避免玩家完全支配市场，使经营玩法更接近双向博弈。

第五，系统将经济模型与生存模型结合。水、面包、烤肉用于恢复水分值或饱食度，银戒指和黄金用于储值和投机。价格模型根据商品类别设置不同波动强度，库存恢复和买卖价差又形成交易摩擦。AI 既要生存，也要积累资金；玩家既要盈利，也要控制 AI 的生存和胜利压力。

第 2 章 系统分析

本章从产品定位、用户价值、运行成本、需求边界和可行性角度分析系统建设条件。系统被定位为一款可继续产品化的 AI 原生经营模拟原型：短期目标是完成可运行的单机闭环，验证玩家商店经营与 AI Agent 行为之间的互动价值；中期目标是形成可扩展的角色、商品、事件和市场参数体系；长期目标是为 AI NPC 驱动的经营模拟游戏提供可复用的架构基础。因此，本章分析重点放在落地运行、交互体验、内容扩展、成本控制和可维护性上。

2.1 设计系统时考虑的相关因素

社会因素方面，生成式人工智能正在快速进入游戏开发和交互娱乐领域。AI NPC 能够增强角色行为多样性，但也可能带来内容不可控和价值表达不稳定的问题。相关综述已经将智能体规划、决策执行和评估视为大模型应用落地的重要问题[2,17]。本系统采用结构化动作与规则校验方式限制模型输出的执行范围，避免 AI 直接生成影响法律、隐私或攻击性内容的行为。系统的主要目标是形成可运营、可调试、可控边界清晰的经营模拟原型，而不是让模型自由生成无边界剧情。

健康因素方面，系统是桌面端二维游戏，操作以鼠标和键盘为主。设计中应避免过强闪烁、长时间高频提示和强制连续操作。回合制机制天然降低了实时反应压力，玩家可以在商店阶段阅读商品价格、AI 状态和资金变化后再做决策。后续若进行完整产品化，应加入暂停、音量控制、字体大小调整和色彩可读性设置。

安全因素方面，系统涉及 OpenAI API 调用和本地 WebSocket 通信。API Key 不应写入仓库或提交到版本管理，应通过环境变量配置。WebSocket 默认绑定本地地址，减少外部访问风险。后端对 Unity 回传的库存、价格和资金信息进行类型转换和边界处理，防止异常数据导致运行时崩溃。动作执行层对地点、库存、背包和资金进行校验，防止模型或前端提交非法状态变更。

法律因素方面，系统使用的部分像素瓦片素材来自公开素材包，项目 README 已注明素材来源和许可信息。论文和系统展示中应保留素材作者、素材地址和使用范围说明。模型服务调用也应遵守对应平台服务条款，不在提示词中包含个人隐私、敏感数据或违法内容。

文化因素方面，像素风小镇经营题材容易被不同用户理解，低门槛且具有较强可视化表达。系统中的 AI 角色身份包括平民、手艺人、银行家、商人等，能够形成具有差异化的经营生态。后续扩展时可以加入更丰富的本土化叙事、节日事件、职业技能和地域文化元素，但应避免刻板化表达。

环境因素方面，系统主要运行在个人计算机上。Unity 前端和 Python 后端资源占用相对可控，二维像素风素材体积较小，不需要高性能图形硬件。大模型调用依赖外部服务，会产生网络流量和计算资源消耗，因此系统在设计上应尽量减少不必要的重复调用，通过计划复用、反思间隔和动作重试上限控制调用频率。

2.2 项目成本分析

项目开发成本主要包括人力成本、软件工具成本、模型调用成本和素材成本。人力成本是主要成本，涉及需求梳理、后端运行时、Unity 前端、UI 资源整合、通信协议、测试和论文撰写。软件工具方面，Python、Visual Studio Code、Unity Personal、Git 等均可免费用于毕业设计开发。模型调用成本取决于计划、行动、反思和决策点调用频率，系统通过不同阶段使用不同模型、结构化输出和错误重试上限控制费用。

项目实施成本主要包括运行环境配置和本地调试成本。DecisionLayer 需要 Python 运行环境、依赖包和 OpenAI API Key；ExecutionLayer 需要 Unity 编辑器和对应项目资源。两层同时运行时需要注意启动顺序，通常先启动 Python 后端，再打开 Unity 场景运行客户端。WebSocket 默认地址为 ws://127.0.0.1:9876，部署时需要保持前后端配置一致。

项目维护成本主要来自三类变化。第一类是模型接口和模型名称变化，需要维护 OpenAIModel 适配逻辑。第二类是 Unity 资源和 UI 变化，可能影响商品图标、回合动画、商店面板和状态弹窗。第三类是游戏规则变化，例如新增商品、调整价格参数、改变胜利条件或加入新动作，需要同步数据文件、动作 handler、validator 和提示词内容。

项目经济效益主要体现在作品展示、技术验证和后续扩展价值。作为毕业设计，系统能够展示 AI Agent、Unity 游戏开发、通信协议和软件工程能力；作为原型项目，系统可以继续扩展成 AI NPC 经营模拟、教学演示或互动叙事实验平台。由于核心依赖是本地 Unity 和 Python，除模型调用外没有复杂服务器成本，适合小规模展示和迭代。

表 2.1 给出项目成本与收益分析。

成本或收益项	具体内容	影响
开发成本	后端、前端、通信、UI、测试与论文	主要成本，集中在毕业设计周期
工具成本	Unity、Python、编辑器、版本管理	基本可免费使用
模型成本	计划、行动、反思和决策点调用	需要通过调用频率控制
维护成本	模型接口、资源、规则和协议变更	中等，可通过分层降低影响
展示收益	体现游戏开发与智能体系统能力	对毕业设计和作品展示价值较高
扩展收益	可扩展商品、事件、角色和玩法	后续研究空间较大

2.3 需求分析

系统用户主要包括玩家、开发者和答辩展示观众。玩家需要通过商店界面完成经营操作，并观察 AI 角色行为和市场变化；开发者需要能够方便地维护角色、商品、地点、事件、动作和市场规则；展示观众需要从界面和论文中理解系统创新点、架构和运行效果。

2.3.1 功能需求分析

玩家商店经营需求包括商品列表展示、商品价格展示、商品库存展示、当日价格修改、进货数量调整、玩家现金显示、回合开始提示、回合结束收益提示和提交经营方案。商品至少包括瓶装水、面包、烤肉、银戒指和黄金。玩家提交后，系统应把 currentMoney、currentStock 和 todayPrice 回传后端。

AI 智能体需求包括角色基础属性、身份描述、资金、位置、背包、饱食度、水分值、精神值、记忆、计划、动作和反思。AI 角色应能根据观察快照决定移动、购买、出售、使用物品、睡觉或等待。系统当前包含四名 AI 角色，并为部分角色保留技能机制。

市场系统需求包括商品基础价格、当前价格、库存、卖出比例、每日价格生成、价格情报、锁价效果和库存同步。价格生成需要保持正数并围绕基础价格回归，不同商品类别具有不同波动参数。玩家对价格的修改应受 AI 锁价影响，被锁定商品当日价格不被玩家覆盖。

动作执行需求包括动作注册、动作别名、动作参数归一化、动作合法性校验和动作结果返回。购买动作需要校验角色位于市场、市场库存足够、角色资金足够；出售动作需要校验角色位于市场、背包物品足够、商店资金足够；使用动作需要校验背包物品；睡觉动作需要校验角色在家。

通信需求包括 Unity 连接后端、后端推送市场信息、后端发送角色动作命令、Unity 回传动作完成消息、Unity 提交商店库存与价格更新、后端等待玩家提交后继续运行。消息格式应采用 JSON，动作命令应携带 action_id，用于匹配请求和回执。

展示需求包括二维小镇场景、角色实体移动、商品交易反馈、状态弹窗、商店库存界面、商品图标映射、回合开始动画、回合结束收益动画和基本 HUD（Head-Up Display，抬头显示）。界面应突出经营信息，避免让玩家在大量调试文本中寻找关键状态。

2.3.2 非功能需求分析

可靠性需求方面，系统应避免因模型输出格式错误、非法动作、Unity 未连接或消息超时而整体崩溃。后端对动作生成失败、无合法动作、未知动作、执行失败、重复动作和交易震荡均需要处理。WebSocket 等待动作回执应设置超时时间，超时后返回失败结果并触发重规划。

可维护性需求方面，角色、商品、地点和事件应尽量通过 CSV（Comma-Separated Values，逗号分隔值）或 JSON 数据文件维护。动作空间应通过注册表扩展，新增动作时不应大量修改运行时主循环。市场参数应集中配置，例如 KAPPA、SIGMA、默认库存、胜利条件和属性消耗值。

可扩展性需求方面，系统后续应能扩展更多地点、商品、角色技能、事件 hook、UI 面板和智能体动作。DecisionLayer 与 ExecutionLayer 的分层为扩展提供基础：新增后端规则不一定要求立即修改 Unity 场景，新增前端表现也不一定改变智能体决策逻辑。

实时性需求方面，系统是回合制经营游戏，不要求毫秒级实时反馈，但角色动作、商店提交和回合切换需要在玩家可接受时间内完成。模型调用是主要耗时来源，因此应限制每回合调用次数，并在 UI 中用回合过渡降低等待感。

可解释性需求方面，系统应保留角色计划、动作结果、反思和市场状态，便于开发者理解 AI 行为原因。动作失败时需要向模型提供明确错误提示，例如地点不符、库存不足、资金不足或重复交易，避免模型机械重复无效行为。

2.4 可行性分析

2.4.1 技术可行性分析

从技术栈看，Unity 能够稳定实现二维场景、角色动画和 UI，Python 能够实现异步运行时、模型调用和数据处理，WebSocket 能够实现双向通信。项目中已经形成 DecisionLayer 和 ExecutionLayer 两个目录，后端包含 actions、config、data、model、runtime 等模块，前端包含 Connector、Player、UI、Camera、Animations 等脚本目录。这说明系统具备清晰的工程基础。

从智能体实现看，系统不依赖完整 Agent 框架，而是自行实现 plan、act、execute、reflect 主循环。该方式适合毕业设计，因为核心逻辑可控，论文可以清楚解释运行时如何组织观察、计划、动作校验和反思。模型调用使用结构化 JSON 输出后，ActionExecutor 再进行确定性检查，技术风险低于让模型直接生成任意脚本。

从经济系统实现看，商品数据、价格参数、库存、买卖价差和属性恢复效果已经具备。均值回归模型计算简单，不需要训练数据；玩家商店阶段通过 WebSocket 等待 Unity 提交数据，能自然插入回合流程。因此经济玩法具备技术可行性。

从风险看，主要技术风险包括模型接口变化、Unity 客户端未连接、WebSocket 消息丢失和模型输出不符合预期。系统已通过 pending 表、action_id、超时处理、动作重试和强制重规划降低风险。后续仍需要增加更多日志和自动化测试，以提高展示稳定性。

2.4.2 操作与维护可行性分析

操作可行性方面，玩家只需要在 Unity 商店界面查看商品信息、调整价格和进货数量，并观察 AI 行为反馈。回合制流程降低了操作复杂度。对于展示场景，开发者可以预先启动后端和 Unity，进入商店阶段后演示玩家定价、AI 行动、交易反馈和回合结算。

维护可行性方面，数据驱动设计降低了内容维护难度。商品调整主要修改 item.csv，角色调整主要修改 actor.csv，随机事件调整主要修改 event.json。动作逻辑集中在 handlers、validators 和 action_registry 中，市场结算集中在 LocationState 和 WorldState 中。该结构有利于定位问题。

部署可行性方面，系统主要面向本地运行，不需要复杂云端部署。DecisionLayer 使用 Python 依赖，ExecutionLayer 使用 Unity 项目，二者通过本地端口通信。只要环境变量、依赖和 Unity 场景配置正确，系统即可运行。对于毕业设计答辩，本地演示的可控性高于公网服务部署。

第 3 章 系统概要设计

本章对应系统总体设计部分，重点说明系统架构、模块划分、数据流、通信协议和核心业务流程。系统概要设计遵循“后端规则可控、前端表现清晰、通信协议简单、玩法闭环完整”的原则。

3.1 系统总体架构设计

系统总体架构由决策层内部结构、前后端通信关系两部分组成。服务端维护权威状态、客户端承担交互表现的拆分方式，能够降低复杂交互系统的耦合度；Unity 相关研究也表明，游戏引擎适合承担可视化交互、场景组织和仿真展示任务[24,28-29]。为了便于后续写入 Word 文档并保证图中节点清晰，整体架构拆分为图 3.1 和图 3.2 两个子图。

图 3.1 决策层内部结构图

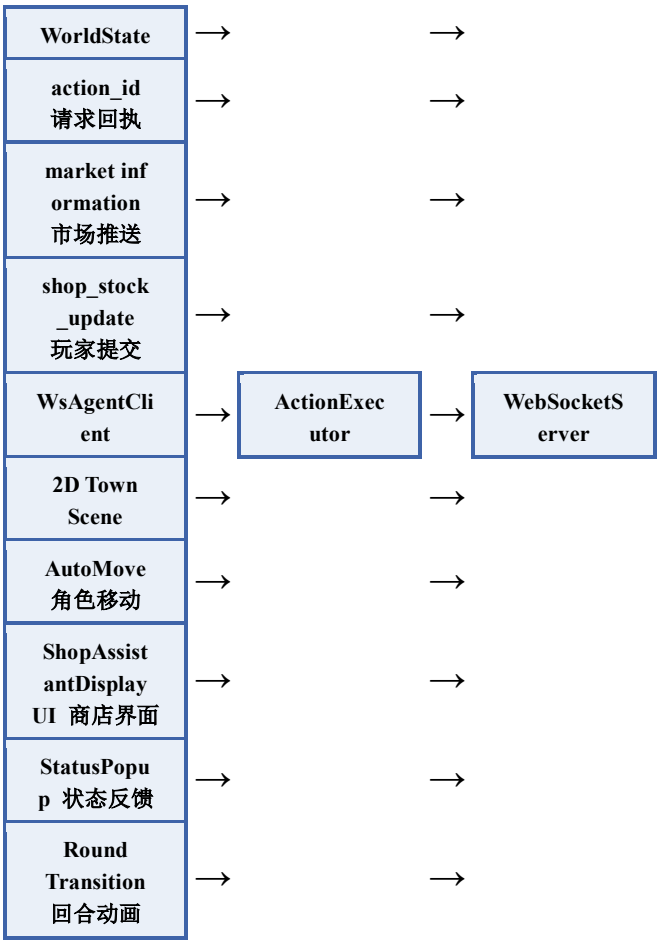

图 3.2 前后端通信与执行层结构图

DecisionLayer 是系统规则中心。WorldState 保存日期、角色、地点、事件、市场和客户端引用；AgentRuntime 负责按 tick 推进每个 AI；PromptBuilder 负责把世界状态翻译成模型可理解的观察上下文；OpenAIModel 负责模型请求；ActionExecutor 负责归一化动作、查找注册表、执行校验器和调用 handler；MarketComponent 负责库存、价格、情报和锁价；MemoryStore 负责记录计划和动作结果。该结构与大语言模型智能体综述中“感知、记忆、规划、行动、评估”的拆分思路一致[17-18]。

Communication 是前后端连接中心。WebSocketServer 监听 Unity 客户端连接，维护 agent_id 与连接对象关系。后端发送动作命令时生成 action_id，将 Future 放入 pending 表；Unity 完成动作后回传 complete，后端根据 action_id 唤醒等待协程。市场信息通过 information 消息广播，玩家经营方案通过 shop_stock_update 回传。

ExecutionLayer 是系统表现中心。Unity 场景中包含小镇地图、角色实体、商店店主、UI 面板、状态弹窗和回合动画。WsAgentClient 负责接收后端消息并转发给导航、动画和商店 UI。ShopAssistantDisplayUI 负责展示商品列表、价格、库存、玩家资金和回合信息，并在玩家提交后把 JSON 发送给后端。

3.1.1 决策层与执行层分离设计

决策层与执行层分离的核心目的是降低耦合。AI 行为、市场模型和胜负规则本质上属于游戏逻辑，适合放在后端集中处理；角色移动、像素素材、动画、按钮和弹窗本质上属于表现逻辑，适合放在 Unity 中处理。若所有逻辑都写入 Unity，则模型调用、异步调度、数据文件加载和动作校验会与场景对象混合，维护难度较高；若所有表现都交给 Python，则游戏体验和像素风展示能力不足。

在当前架构中，后端不需要知道 Unity 中角色具体使用哪一个 Animator，也不需要直接操作 Transform。它只发送“移动到集市”“购买商品”“显示状态变化”等命令。Unity 接收到命令后，根据场景中的导航目标、角色对象和 UI 资源执行具体表现。反过来，Unity 不需要理解 AI 的提示词和计划逻辑，它只需要把动作执行结果和玩家商店提交结果回传后端。

该分离带来三个好处。第一，后端可以在无 Unity 的情况下进行部分逻辑测试，例如价格生成、动作校验和数据加载。第二，前端可以在 mock 数据下调试商店 UI、回合动画和商品图标。第三，通信协议稳定后，前后端可以分别扩展，不会因一个界面资源改动导致整个智能体运行时重构。

3.1.2 通信协议与回合闭环设计

通信协议采用 JSON 消息。主要消息类型包括 hello、hello_ack、information、command、animation、complete、shop_stock_update、ping 和 pong。Unity 启动后发送 hello，后端记录连接并返回 hello_ack。后端推送市场信息时发送 information，target 为 market。后端要求 Unity 执行动作时发送 command 或 animation，并附带 action_id。Unity 执行完成后发送 complete，status 为 ok 表示成功。

表 3.1 给出主要通信消息。

消息类型	发送方向	主要字段	功能
hello	Unity 到 Python	agent_id 或	建立客户端绑定

		agent_ids	
information	Python 到 Unity	target、info	推送市场和玩家信息
command	Python 到 Unity	cmd、target、value、action_id	控制移动、回合开始、回合结束等
animation	Python 到 Unity	target、value、action_id	展示属性、金钱和物品变化
complete	Unity 到 Python	action_id、status	回传动作执行结果
shop_stock_update	Unity 到 Python	info	提交玩家价格、库存和资金

回合闭环由商店阶段、AI 行动阶段和日结算阶段组成。商店阶段由后端推送商品信息并发送 round_start 命令，Unity 展示回合开始动画和商店面板。玩家提交后，后端将价格和库存写入 MarketComponent。AI 行动阶段由 AgentRuntime 调度每名 AI 执行计划与动作。日结算阶段在所有 AI 输出 wait 后触发，系统发送 round_end，推进日期，生成下一日价格，刷新 AI 状态和决策点，并再次进入商店阶段。

3.2 功能模块设计

系统功能模块拆分为总体模块、决策与经济模块、前端与支撑模块三部分，分别如图 3.3、图 3.4 和图 3.5 所示。

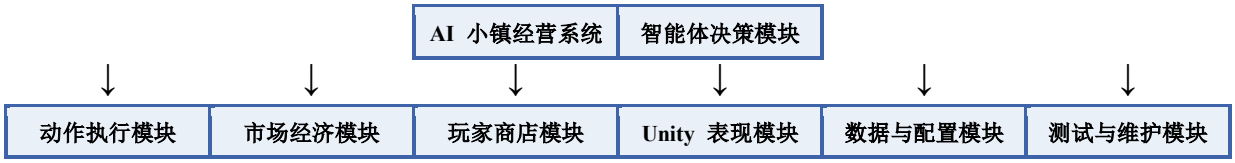

图 3.3 系统功能模块总览图

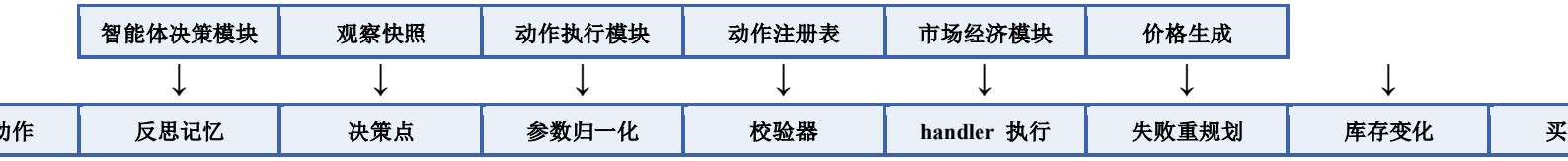

图 3.4 决策、动作与市场模块图

图 3.5 前端、数据与维护模块图

智能体决策模块负责把角色当前状态、市场状态、事件、记忆和决策点上下文组织成观察快照，再调用模型生成计划和动作。动作执行模块是模型输出进入游戏世界的关口，负责确保动作合法。市场经济模块负责商品价格、库存、交易摩擦和 AI 反制信息。玩家商店模块是玩家主要交互入口。Unity 表现模块提供场景、动画、UI 和反馈。数据与配置模块支持内容维护。测试与维护模块负责连通性测试、回归检查和日志排查。

系统数据流如图 3.6 所示。

步骤	发起方	接收方	消息
1	Python 决策层	WebSocket 通信	推送市场信息
2	WebSocket 通信	Unity 执行层	information market
3	Unity 执行层	玩家	展示商店界面
4	玩家	Unity 执行层	调整价格与进货
5	Unity 执行层	WebSocket 通信	shop_stock_update
6	WebSocket 通信	Python 决策层	写入市场状态
7	Python 决策层	LLM 模型	发送观察上下文
8	LLM 模型	Python 决策层	返回计划或动作
9	Python 决策层	Python 决策层	动作校验与状态更新

10	Python 决策层	WebSocket 通信	command 或 animation
11	WebSocket 通信	Unity 执行层	执行动作
12	Unity 执行层	WebSocket 通信	complete
13	WebSocket 通信	Python 决策层	唤醒等待协程
14	Python 决策层	Python 决策层	回合结算

图 3.6 系统数据流图

第 4 章 系统详细设计与实现

本章对应系统实现部分，重点说明 DecisionLayer、市场系统、玩家商店系统、Unity 执行层和稳定性机制的详细设计。为了满足论文代码篇幅限制，本章以结构、流程和关键逻辑说明为主，不连续展示大段源代码。

4.1 决策层详细设计与实现

DecisionLayer 目录承担系统后端功能。其主要子目录包括 actions、config、data、model 和 runtime。actions 存放动作注册、校验和执行逻辑；config 存放市场参数、模型名称、胜负条件、属性衰减和运行时阈值；data 存放角色、商品、地点和事件数据；model 存放世界状态、角色状态、地点状态、定义类和大模型封装；runtime 存放数据加载、世界构建和 Agent 调度。

WorldState 是世界级动态状态容器。它维护 day、events、actors、locations、client 和 shop_assistant_last_money。update_day 方法负责判断是否所有 AI 都结束本回合，若仍有角色 running 为 true，则不推进日期；若所有 AI 均结束，则发送回合结束反馈、推进 day、进入玩家商店阶段、刷新角色每日状态、判定随机事件并执行每日结算 hook。该设计保证回合推进由所有智能体共同决定，而不是由单个角色单独推进。

ActorState 是角色级动态状态。它包含资金、位置、家、记忆、属性、背包、已知地点、事件、运行状态、决策点和决策点上下文。属性包括 hunger、thirst 和 fatigue。角色每日更新时恢复 running、追加记忆记录、恢复决策点，并按配置衰减生存属性。角色还提供 apply_decision_point 和 apply_decision_point_batch，用于执行 AI 的锁价、情报和现金兑换结果。

LocationState 管理地点动态组件。当前核心地点是 market，内部挂载 MarketComponent。MarketComponent 维护库存、当前价格、下一日价格、锁价集合和当日锁定集合，并提供价格生成、库存增减、情报价格模拟、玩家商店提交应用等方法。

4.1.1 多智能体运行时设计

AgentRuntime 负责多智能体调度。每个 AI 在运行时都有 RuntimeActorState，记录 step、plan_id、当前计划、最近结果、是否需要重规划、重复动作统计、交易方向统计、拆分动作统计和决策点执行状态。运行时主流程可以概括为观察、决策点、计划、行动、校验保护、执行、记忆、反思。该流程吸收了 ReAct 对“推理与行动交替”的思想[3]，也吸收了 Reflexion 和智能体记忆机制研究中关于失败反馈、历史经验和后续决策关联的思路[4,18]。

观察阶段由 world.observe 生成。观察快照包括角色身份、当前地点、资金、属性、背包、市场价格、库存、工作事件、世界事件、角色 buff、记忆、当前计划、历史计划和决策点私有上下文。PromptBuilder 使用这些信息构造 plan、act、reflect 三类提示词。

决策点阶段发生在每个角色每天第一次行动前。智能体可以选择 skip、exchange_cash、get_intel 或 lock_price。系统真正执行决策点效果时才扣除资源或修改市场，而不是让模型输出直接生效。get_intel 会生成最多三个商品的价格情报，包括当前价格、情报价格、

趋势、准确率和是否有效。lock_price 会锁定指定商品下一日价格，并使相同商品的情报失效。

计划阶段由模型根据观察生成本回合目标和策略。行动阶段要求模型输出结构化动作。运行时支持对模型输出进行归一化，例如兼容 type、name、cmd、action_type 等字段，并将 item_id、destination、location 等字段转换为统一动作格式。该设计提高了模型输出的容错性。

执行前保护机制包括重复动作保护、交易震荡保护和拆分动作保护。重复动作保护避免智能体连续执行完全相同动作；交易震荡保护避免同一回合对同一商品先买后卖或连续重复交易；拆分动作保护避免模型把同类同物品动作拆成多次小动作，从而造成不必要的精神消耗。若处于生存危机，系统允许重复 consume 或 sleep，以免保护机制阻断必要自救。

反思阶段根据执行结果、失败情况和时间间隔触发。反思文本进入后续提示上下文，使智能体能够利用失败反馈调整策略。该机制与 Reflexion 的语言反思思想一致，但在本系统中反思只作为提示上下文，不修改模型参数。

4.1.2 动作空间与校验机制设计

动作空间采用注册表设计。每个动作由动作名、handler 和 validators 组成。register 装饰器把动作注册到全局表中，resolve_name 支持动作别名，get_entry 根据动作名获取入口。ActionExecutor 执行时先归一化动作参数，再查询注册表，依次执行 validators，全部通过后调用 handler。机器人和具身智能体相关研究强调，语言模型计划必须受到环境可执行性和技能边界约束[11,13]，因此本系统没有让模型直接修改世界状态，而是把动作空间显式化。

系统当前核心动作如表 4.1 所示。

动作	含义	主要校验	状态影响
move	移动到目标地点	目标不能等于当前位置	修改位置，消耗精神值
buy	在市场购买商品	位于市场、库存足够、资金足够	增加背包，减少资金，减少市场库存
sell	在市场出售商品	位于市场、背包足够、商店资金足够	减少背包，增加资金，增加市场库存
consume	使用消耗品	背包有足够物品	恢复饱食度或水分值，消耗精神值
sleep	在家睡觉	位于家	恢复精神值，消耗饱食度和水分值
wait	结束回合	无特殊校验	角色 running 置为 false，尝试推进日期

动作校验机制的价值在于把模型输出变成待审查提案。以 buy 动作为例，模型可能输出购买黄金，但若角色不在市场、市场库存不足或角色资金不足，则校验器返回失败

结果，运行时把错误提示写入 prompt_builder.error_log 并强制重规划。模型下一次 act 时会看到具体失败原因，从而改用移动、减少数量或结束回合。

动作 handler 不仅修改后端状态，还通过 WebSocket 请求 Unity 执行表现。例如 buy 会要求 Unity 展示物品增加、移动到收银台和金钱减少；sell 会展示移动到收银台、金钱增加和物品减少；consume 会展示物品减少和属性恢复。Unity 返回成功后，后端才更新资金、库存和背包，保持前后端行为一致。

4.2 市场与玩家商店系统实现

市场系统是本项目从“AI 自循环交易模拟”升级为“玩家经营博弈”的关键。商品数据保存在 item.csv 中，字段包括 itemId、name、category、purchasePrice、basePrice、sellRatio、quantity、description、hunger、thirst 和 fatigue。当前商品包括瓶装水、面包、烤肉、银戒指和黄金。

价格模型采用对数空间的离散均值回归：

$$\log P_{t+1}=\log P_t+\kappa\left[\log P^*-\log P_t\right]+\varepsilon_t$$

$$\varepsilon_t\sim N\left[0,\sigma^2\right]$$

其中，P_t 为当前价格，P^* 为基础价格，κ 为回归速度，σ 为波动强度。系统中消耗品和贵重品的 KAPPA 均为 0.11，表示价格每天向基础价格回归约 11%。消耗品 SIGMA 为 0.15，贵重品 SIGMA 为 0.4。该参数使水、面包、烤肉保持相对稳定，而银戒指和黄金具有更明显投机属性。

玩家商店阶段发生在每日开始。后端调用 client.market_information 生成商品列表和玩家资金信息，再通过 information 消息推送给 Unity。Unity 的 ShopAssistantDisplayUI 解析市场信息，动态生成商品格子，展示商品图标、名称、基础价格、当日价格、库存和锁价状态。玩家可以修改商品价格和进货数量，提交后 Unity 通过 shop_stock_update 发送 JSON 到后端。

后端 MarketComponent.apply_shop_stock_update 根据玩家提交结果更新市场库存、当日价格和玩家资金。如果某商品处于 locked_today_items 集合中，则玩家提交的 todayPrice 不会覆盖该商品价格。这样 AI 的锁价决策能够真实限制玩家市场操作。

AI 与商店的资金关系通过 human shop assistant actor 表示。AI 购买商品时，AI 资金减少，商店资金增加；AI 出售商品时，AI 资金增加，商店资金减少。若商店资金不足，出售动作会被拒绝。该设计让玩家现金流不只是 UI 数字，而是参与市场交易的真实约束。

玩家胜负逻辑围绕 WIN_CONDITION 展开。玩家资金达到 10000 时胜利；玩家破产时失败；任一 AI 死亡或任一 AI 先达到 10000，也意味着玩家未能维持小镇平衡。该目标设计使玩家不能单纯提高所有商品价格，也不能无限低价让 AI 获利。

4.3 Unity 执行层详细设计与实现

ExecutionLayer 是 Unity 项目目录，核心脚本位于 Assets/Scripts。Connector/WsAgentClient.cs 负责 WebSocket 客户端通信、消息路由、动作执行和回执发送；UI/ShopAssistantDisplayUI.cs 负责商店经营界面、商品格子、价格与库存调整、回合开始和回合结束动画；Player/AutoMove.cs 和 ClickToMove2D.cs 负责角色移动；UI/StatusPopupSpawner.cs 和 PopItem.cs 负责状态弹窗；Camera/PixelGridCameraSnapRuntime.cs 负责像素网格相机对齐。已有 Unity 游戏实践通常围绕场景、角色、交互界面和反馈动画组织客户端模块[23-27]，本系统在此基础上进一步接入后端智能体决策层。

WsAgentClient 接收到 information 且 target 为 market 时，会把 info 转交给 ShopAssistantDisplayUI.PushMarketInformationJson。接收到 command 且 cmd 为 round_start 或 round_end 时，会调用商店 UI 展示回合过渡。接收到 go_to 时，会通过导航组件控制角色移动到目标地点。对于非移动命令和 animation 消息，客户端会触发相应状态动画和反馈，然后通过 SendJsonShared 回传 complete。

ShopAssistantDisplayUI 以动态构建方式生成商店面板。它从后端 market information 中解析商品信息，并结合 Resources 中的商品图标映射显示不同商品。界面包含回合数、资金、商品列表、进货按钮、库存调整控件、价格输入或步进控件，以及提交成功提示。回合开始时先展示过渡动画，再打开库存操作界面；回合结束时展示当日收益变化。

Unity 表现层的设计重点是让玩家直观看到经营信息和 AI 行为。商品图标帮助玩家识别水、面包、烤肉、银戒指和黄金；状态弹窗显示金钱、物品、饱食度、水分值和精神值变化；角色移动让 AI 行动不只停留在日志文本；回合动画让商店阶段和 AI 行动阶段有明确边界。

表 4.2 给出 Unity 主要脚本职责。

脚本	所属模块	主要职责
WsAgentClient.cs	Connector	WebSocket 连接、消息解析、命令路由、回执发送
ShopAssistantDisplayUI.cs	UI	商店界面、商品列表、价格库存提交、回合动画
AutoMove.cs	Player	自动寻路与角色移动表现
ClickToMove2D.cs	Player	玩家点击移动辅助
PlayerHUD.cs	Player	角色状态信息展示
StatusPopupSpawner.cs	UI	状态变化弹窗生成
DialogueBubbleUI.cs	UI	对话气泡展示

PixelGridCameraSnapRuntime.cs	Camera	像素风相机对齐

4.4 系统稳定性与异常处理

LLM 智能体系统的稳定性难点主要来自模型输出不稳定、外部执行不确定和多主体状态相互影响。系统针对这些问题设计了多层保护。

第一层是结构化输出保护。act 阶段要求模型输出 JSON 动作对象，运行时再通过 _coerce_proposal 兼容不同字段形式。若模型输出不是字典、列表中没有动作或缺少动作类型，系统返回 NO_ACTION 并触发重规划。

第二层是动作合法性保护。ActionExecutor 对未知动作返回 INVALID_ACTION，对缺少动作名返回 INVALID，对地点、库存、背包、资金等条件不满足的情况返回具体失败信息。失败信息进入下一次提示词，指导模型修正。

第三层是行为循环保护。RuntimeActorState 记录上一动作签名、同类交易统计和最近成功动作。若检测到重复动作、交易震荡或拆分动作，系统拒绝执行并要求重规划。该保护对于长期运行智能体非常重要，因为模型可能在局部目标下重复购买同一商品或在买卖之间震荡。

第四层是通信超时保护。WebSocketServer 发送动作后把 Future 存入 pending 表，并使用 wait_for 设置超时时间。若 Unity 未回传 complete，后端移除 pending 并返回失败，避免协程永久挂起。连接断开时，server 会清理对应 agent_id 和连接对象。

第五层是市场数据容错。玩家提交数据时，后端会解析 parsed_info，并对 currentMoney、currentStock 和 todayPrice 做类型转换和边界处理。库存取不小于 0 的整数，价格取不小于 0 的浮点数。锁价商品不允许被玩家提交覆盖。

第六层是配置集中管理。胜利条件、属性衰减、市场波动、库存恢复、模型名称、动作重试次数和保护阈值都集中在 config.py 中。集中配置有利于调参和测试，也减少魔法数字散落在业务代码中的风险。

第 5 章 系统部署与维护

本章对应系统部署、测试和维护部分。由于系统由 Python 后端和 Unity 前端共同组成，部署过程需要确保依赖、环境变量、端口和启动顺序一致。

5.1 系统安装与部署

5.1.1 决策层部署

DecisionLayer 部署需要 Python 3.11 及以上环境。进入 DecisionLayer 目录后，安装 requirements.txt 中的依赖。运行前需要配置 OPENAI_API_KEY 环境变量。Windows PowerShell 可使用环境变量设置命令，macOS 或 Linux 可使用 export 命令。后端入口是 main.py，建议以 DecisionLayer 作为工作目录运行，以保证 data 目录下的 actor.csv、item.csv、location.csv 和 event.json 能被正确读取。

后端启动后会构建 Catalog、WorldState、WebSocketServer、AgentRuntime 和 ActionExecutor。WebSocket 默认监听 127.0.0.1:9876。若 Unity 尚未连接，后端会等待客户端连接。部署时应确认防火墙或端口占用不会阻止本地连接。

5.1.2 执行层部署

ExecutionLayer 使用 Unity 打开。打开项目后，需要确认场景中 WsAgentClient 的连接地址与后端一致，默认地址为 ws://127.0.0.1:9876。运行顺序通常为先启动后端，再运行 Unity 场景。Unity 启动后发送 hello，后端返回 hello_ack，并推送市场信息。若商店界面未显示，应检查 Unity 控制台是否收到 information market 消息。

Unity 侧还需要确认商品图标、商店背景、按钮资源和回合动画资源是否位于预期 Resources 或 Assets 路径。由于素材授权限制，仓库可能不包含原始素材文件，因此在展示环境中应提前检查资源是否完整。

5.2 系统测试

系统测试应覆盖后端逻辑、前后端通信、玩家商店阶段和完整回合流程。由于多智能体系统容易受到非平稳环境、主体间相互影响和通信协作复杂性的影响[19-21]，测试不能只验证单个函数正确性，还需要验证回合推进、市场反馈和多主体动作序列是否能够稳定闭环。

后端逻辑测试包括数据加载、市场价格生成、库存增减、动作校验、角色属性更新、决策点执行和回合推进。可通过构造 WorldState 和 MarketComponent 的方式检查价格是否为正、库存是否不为负、锁价是否覆盖玩家提交、情报是否在锁价后失效。

通信测试包括 Unity 连接、hello_ack、information 推送、command 发送、complete 回执和 shop_stock_update 提交。WebSocketServer 中已经包含部分 smoke test 思路，例如发送 market information、round_start 并等待玩家提交。

功能测试包括以下用例。

测试编号	测试内容	预期结果
T1	启动后端并运行 Unity	Unity 连接成功，后端记录 agent_id
T2	推送市场信息	商店界面展示五类商品和玩家资金
T3	玩家修改价格并提交	后端更新非锁价商品当日价格
T4	AI 执行 buy	Unity 展示交易反馈，后端更新资金和库存
T5	AI 执行 sell	商店资金足够时交易成功，不足时返回失败
T6	AI 执行 consume	背包减少，属性恢复，弹窗显示
T7	所有 AI wait	系统进入回合结算并开启下一回合
T8	AI 锁定商品价格	玩家提交不覆盖锁价商品价格
T9	Unity 不回传 complete	后端超时并返回失败，不永久阻塞
T10	模型输出未知动作	ActionExecutor 返回 INVALID_ACTION 并触发重规划

性能测试方面，本系统属于小规模回合制项目，主要瓶颈是模型调用和 Unity 动画等待。由于角色数量为四名，动作间隔为秒级，系统在普通个人计算机上可以完成展示。若未来增加角色数量，应考虑批量调度、减少反思频率、限制每回合最大动作数和缓存部分提示上下文。

5.3 系统维护

系统维护需要关注数据、规则、协议、界面和模型五类内容。

数据维护方面，角色、商品、地点和事件应保持字段完整。新增商品时需要在 item.csv 中提供 category、basePrice、sellRatio、quantity 和属性恢复值，并在 Unity 侧配置图标映射。新增角色时需要在 actor.csv 中提供 actorId、name、gender、age、info 和 skill。

规则维护方面，新增动作应先设计动作语义，再添加 handler 和 validators，最后更新 PromptBuilder 的动作说明。市场规则变化应集中修改 config.py 和 MarketComponent，避免在多个脚本中重复硬编码。

协议维护方面，新增 Unity 命令应同时更新 WebSocketServer 和 WsAgentClient。消息字段应尽量保持向后兼容，例如新增字段不应影响旧字段解析。action_id 必须在需要回执的命令中保留，否则后端无法匹配 complete。

界面维护方面，ShopAssistantDisplayUI 文件较大，应在后续迭代中按商品格子、回合过渡、资源加载、提交逻辑等职责拆分，以降低维护难度。若继续扩展经营面板，应保持信息层级清晰，避免把调试信息直接暴露给玩家。

模型维护方面，需要关注模型名称、接口参数和结构化输出方式变化。若模型切换导致动作输出格式变化，应优先调整 PromptBuilder 和 _coerce_proposal，而不是放宽动作执行边界。模型错误应通过日志记录，便于复现。

5.4 项目管理与迭代

项目迭代可分为四个阶段。第一阶段完成基础小镇、角色移动、WebSocket 通信和动作执行。第二阶段完成多智能体计划、行动、记忆和反思闭环。第三阶段完成市场经济系统、玩家商店阶段、价格模型和库存同步。第四阶段完成 AI 决策点、回合动画、商店 UI 优化和论文整理。

后续迭代方向包括以下内容。

第一，扩展角色差异。当前角色主要通过身份文本、初始资金和技能字段区分，后续可加入更明确的风险偏好、消费习惯、职业目标和社交关系。

第二，扩展商品和地点。当前地点只有家和集市，商品数量较少。可加入医院、餐馆、仓库、银行等地点，以及药品、工具、奢侈品等商品，使经营策略更丰富。

第三，扩展事件系统。随机事件目前包括下雨天和过路费等简单效果。后续可加入节日需求、供应短缺、市场谣言、突发疾病等事件，与价格和生存系统联动。

第四，扩展测试工具。可增加无 Unity 模式下的模拟客户端，用于自动跑回合并统计 AI 生存率、玩家收益和市场波动，辅助参数调优。

第五，优化 UI 结构。当前商店 UI 已实现核心功能，但脚本体量较大。后续应拆分为数据模型、商品格子组件、提交控制器和过渡动画控制器，提高可读性。

结论

论文围绕基于大语言模型智能体的 Unity 二维像素风小镇经营系统进行了设计与实现。系统采用 DecisionLayer 与 ExecutionLayer 双层架构，后端负责智能体决策、世界状态、动作校验、市场结算和回合推进，前端负责二维场景、角色动画、商店界面和玩家操作。两层通过 WebSocket 长连接交换 JSON 消息，形成可运行的玩家商店阶段、AI 行动阶段和回合结算闭环。

在智能体设计方面，系统实现了计划、行动、执行、反思的运行时流程，并通过动作注册表、校验器、重复行为保护、交易震荡保护和通信超时处理提高稳定性。模型输出不直接修改世界，而是作为动作提案进入确定性规则系统，从而兼顾 LLM 行为灵活性和游戏状态安全。

在经营系统设计方面，系统把玩家设置为小镇唯一商店店主，使玩家通过定价、进货和现金流管理影响 AI 生存和收益。市场价格采用对数空间的均值回归模型，并按商品类别设置不同波动强度。AI 决策点机制进一步提供情报、锁价和现金兑换能力，使 AI 能够对玩家经营策略产生反制。由此，系统从单纯 AI 观察模拟转变为人类玩家与智能体共同参与的动态经营博弈。

在工程实现方面，系统完成了 Python 后端、Unity 前端、WebSocket 通信、商品数据、角色数据、市场信息推送、玩家提交、动作执行和回合过渡等核心功能。项目证明了 LLM 智能体与轻量级游戏运行时结合的可行性，也为后续扩展更多地点、商品、事件、角色技能和测试工具提供了基础。

系统仍存在不足。首先，当前 AI 行为质量依赖模型输出和提示词设计，尚未建立系统化量化评估指标。其次，Unity UI 仍有进一步组件化和重构空间。再次，角色数量、商品种类和地点规模较小，长期经营深度有限。最后，模型调用成本和网络可用性仍会影响演示稳定性。后续工作可以从自动化评估、多回合仿真测试、UI 模块拆分、内容扩展和本地模型适配等方向继续完善。

参考文献

[1] Park J S, O'Brien J C, Cai C J, et al. Generative Agents: Interactive Simulacra of Human Behavior[C]. Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology, 2023.

[2] 郭先会，张梦姣，马军. 基于大语言模型的智能体构建综述[J]. 通信技术, 2024, 57（9）:873-879.

[3] Yao S, Zhao J, Yu D, et al. ReAct: Synergizing Reasoning and Acting in Language Models[C]. International Conference on Learning Representations, 2023.

[4] Shinn N, Cassano F, Berman E, et al. Reflexion: Language Agents with Verbal Reinforcement Learning[EB/OL]. arXiv:2303.11366, 2023. https://arxiv.org/abs/2303.11366.

[5] Wang G, Xie Y, Jiang Y, et al. Voyager: An Open-Ended Embodied Agent with Large Language Models[EB/OL]. arXiv:2305.16291, 2023. https://arxiv.org/abs/2305.16291.

[6] 李明阳，许可儿，宋志强，等. 多智能体强化学习算法研究综述[J]. 计算机科学与探索, 2024, 18（8）:1979-2003.

[7] Wei J, Wang X, Schuurmans D, et al. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models[C]. Advances in Neural Information Processing Systems, 2022.

[8] Yao S, Yu D, Zhao J, et al. Tree of Thoughts: Deliberate Problem Solving with Large Language Models[C]. Advances in Neural Information Processing Systems, 2023.

[9] Schick T, Dwivedi-Yu J, Dessi R, et al. Toolformer: Language Models Can Teach Themselves to Use Tools[C]. Advances in Neural Information Processing Systems, 2023.

[10] 王若男，董琦. 基于学习机制的多智能体强化学习综述[J]. 工程科学学报, 2024, 46（7）:1251-1268.

[11] Ahn M, Brohan A, Brown N, et al. Do As I Can, Not As I Say: Grounding Language in Robotic Affordances[EB/OL]. arXiv:2204.01691, 2022. https://arxiv.org/abs/2204.01691.

[12] Huang W, Xia F, Xiao T, et al. Inner Monologue: Embodied Reasoning through Planning with Language Models[EB/OL]. arXiv:2207.05608, 2022. https://arxiv.org/abs/2207.05608.

[13] Song C H, Wu J, Washington C, et al. LLM-Planner: Few-Shot Grounded Planning for Embodied Agents with Large Language Models[EB/OL]. arXiv:2212.04088, 2022. https://arxiv.org/abs/2212.04088.

[14] Wu Q, Bansal G, Zhang J, et al. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation Framework[EB/OL]. arXiv:2308.08155, 2023. https://arxiv.org/abs/2308.08155.

[15] Guo T, Chen X, Wang Y, et al. Large Language Model Based Multi-agents: A Survey of

Progress and Challenges[C]. Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence, 2024:8048-8057.

[16] 项凤涛, 罗俊仁, 谷学强, 等. 群视角下的多智能体强化学习方法综述[J]. 智能科学与技术学报, 2023, 5（3）:322-335.

[17] Wang L, Ma C, Feng X, et al. A Survey on Large Language Model Based Autonomous Agents[J]. Frontiers of Computer Science, 2024, 18（6）:186345.

[18] Zhang Z, Bo H, Chen C, et al. A Survey on the Memory Mechanism of Large Language Model Based Agents[J]. ACM Transactions on Information Systems, 2025.

[19] Gronauer S, Diepold K. Multi-agent Deep Reinforcement Learning: A Survey[J]. Artificial Intelligence Review, 2022, 55:895-943.

[20] 闫超, 向小佳, 许新, 等. 多智能体深度强化学习及其可扩展性与可迁移性研究综述[J]. 控制与决策, 2022, 37（12）:3039-3052.

[21] 田琪, 杨博, 刘美琴, 等. 通信协作多智能体强化学习研究综述[J]. 控制与决策, 2023, 38（5）:1217-1231.

[22] 薛宏涛, 叶媛媛, 沈林成, 等. 多智能体系统体系结构及协调机制研究综述[J]. 机器人, 2001, 23（1）:85-90.

[23] 叶现勇, 许向荣, 梁启萌. 基于Unity3D的汉字游戏的设计与实现[J]. 现代信息科技, 2022, 6（5）:22-25, 4.

[24] 刘宁晖, 宋瑾钰. 基于 Unity3D 的文字冒险游戏的设计与开发[J]. 软件工程与应用, 2022, 11（4）:779-787.

[25] 秦思, 侯建峰. 基于 Unity3D 的体感游戏设计与实现[J]. 计算机科学与应用, 2016, 6（8）:477-483.

[26] 余欣婷, 曾妮丽, 许晔, 等. 基于 Unity 的三维游戏 APP 设计与实现[J]. 电脑编程技巧与维护, 2020（10）:61-62, 71.

[27] 刘梦颖, 马宏琳. 基于 Unity 的冒险游戏设计与实现[J]. 河南科技, 2021（17）:30-32.

[28] 干建松, 徐梓航, 宗志航, 等. 基于 Unity 的新能源汽车虚拟仿真系统设计与实现[J]. 常州信息职业技术学院学报, 2025（4）.

[29] 俞晓妮, 贾婷君. 基于 Unity 的仿真教学系统的设计与研究[J]. 魅力中国, 2021（16）:366.

[30] 陈卓然, 刘泽阳, 万里鹏, 等. 多智能体强化学习理论及其应用综述[J]. 模式识别与人工智能, 2024, 37（10）:851-872.

致　谢

空2格(1个汉字位)，但在目录里不空格

××××××（文字和段落格式按正文文本）××××××

附　录

空2格(1个汉字位)，但在目录里不空格

××（按正文文字和段落格式）××××××××××××××××××××××××××××××××××××××